

NeurIPS 2022 | 用二阶信息给 Lasso 提速： Hessian 筛选规则

隆德大学提出 Hessian Screening Rule：同一份曲率信息既让特征筛选更紧、又让求解器的热启动几乎精确，在高相关场景下优势最大

Lasso 是高维统计里最可靠的工具之一：在回归损失上加一个 ℓ_1 惩罚，模型就能一边拟合、一边做特征选择。麻烦在于，合适的惩罚强度 λ 事先并不知道。实际做法不是拟合一个 Lasso 模型，而是沿着一整条正则化路径（regularization path），在很多个 λ 上都拟合一遍，再用交叉验证挑最好的一个，同一个昂贵的问题要反复重解。当特征有几万维时，时间几乎都花在这条路径上。

让每次拟合更省的常用技巧，是筛选规则（screening rule）：在求解器运行之前，先廉价地猜出哪些特征的系数会保持为零，把它们提前丢掉，让优化器只面对一个很小的候选集合。筛选规则正是大规模 Lasso 得以落地的关键。但主流筛选规则有个共同的软肋：恰恰在特征高度相关时变得保守，而这正是最需要速度的场景。它们会保留过多特征，交给求解器的热启动（warm start，用上一步的解作为下一步的初值）也不够准，收敛因此变慢。

这篇 NeurIPS 2022 论文由隆德大学的 Johan Larsson 与 Jonas Wallin 完成，把上面两个问题都归到同一个根源（只用了一阶信息），并用同一件工具来修复。Hessian 筛选规则（Hessian Screening Rule）改用二阶（曲率）信息来估计路径的下一步。作者证明，一份 Hessian 可以同时干两件事：既把特征筛得更紧，又给出一个在活跃集不变时几乎就是精确解的热启动。最终在几乎所有模拟和真实数据基准上，它都是最快的方法。

论文：<https://arxiv.org/abs/2104.13026>

代码（C++/R）：<https://github.com/jolars/HessianScreening>

真正的开销：拟合一整条路径

由于 λ 必须调，工作单位不是一次 Lasso 拟合，而是沿着一列递减的惩罚值依次求解，每一步都用上一步的解来初始化。序贯筛选规则，如强规则（strong rule）和工作集策略（working-set strategy），正是利用了这一点：用 λ_k 处的解，去预测在下一个更小的 λ_{k+1} 处哪些特征可能变为活跃。论文的第二个观察带有统一性：这两种规则都可以改写成对同一个量的估计，即下一步的相关性（correlation，也就是负梯度）。这个改写暴露了它们共同的缺陷。这个估计是纯一阶的，一旦特征相关就会明显偏移，逼着求解器做额外的 KKT 检查和重新优化。

一份 Hessian，两个用途

方法建立在一个简单事实上：在任意一段 λ 区间内，只要非零系数构成的活跃集不变，Lasso 的解就是 λ 的线性函数。而线性正是二阶估计能利用的结构。作者用活跃特征的 Hessian，写出下一步相关性的一个更锐利的二阶估计：

$$\hat{c}_H(\lambda_{k+1}) = c(\lambda_k) + (\lambda_{k+1} - \lambda_k) \cdot X^T X_A (X_A^T X_A)^{-1} \text{sign } \beta(\lambda_k)_A$$

为了控制开销，昂贵的内积只在强规则集合上计算，再加回一小部分单位上界作为安全余量。同一份 Hessian 的逆矩阵随后一物两用，直接给系数本身当热启动；而且因为在固定活跃集上解对 λ 线性，这个热启动不只是接近，而是在活跃集不变时就等于精确解：

$$\beta(\lambda_{k+1})_A = \beta(\lambda_k)_A + (\lambda_k - \lambda_{k+1}) \cdot H_A^{-1} \text{sign } \beta(\lambda_k)_A$$

两处工程设计让它在大规模下依然实用：低秩更新（low-rank update）在特征进出活跃集时，实时维护 Hessian 及其逆；近似同伦（approximate homotopy）则自适应地安排 λ 网格。更重要的是，这里没有任何一步依赖平方误差。规则适用于任何二阶可微的光滑损失，因此 ℓ_1 正则的逻辑回归可以顺带覆盖。

用途一：筛选紧得多

第一个收益，是规则保留的特征之少。下图在一个 $n = 200$ 、 $p = 20000$ 的设计上拟合完整路径，取三种相关水平，画出每种规则在每一步筛入（保留为候选）的特征数量；虚线是真实活跃集的下界。注意 y 轴是对数刻度，因此纵向的距离就是数量级的差距。

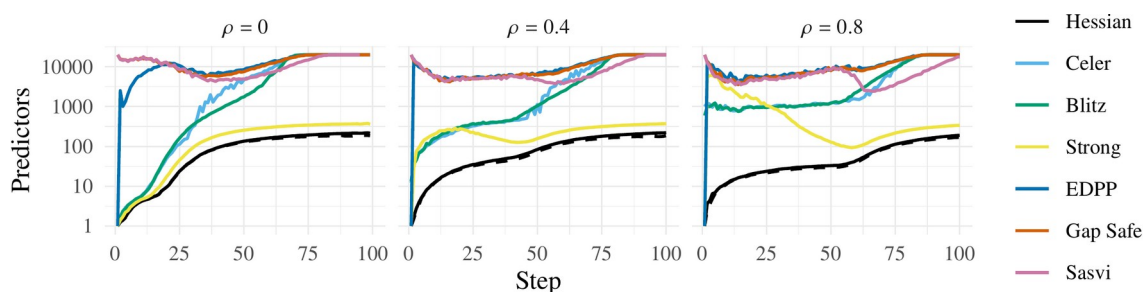


图1 相关水平 $\rho = 0, 0.4, 0.8$ 下每一步筛入的特征数 ($n = 200$, $p = 20000$, 对数刻度, 20 次重复取平均)。Hessian 规则（黑线）紧贴真实活跃集下界（虚线），而 Celer、Blitz、Strong、EDPP、Gap Safe、Sasvi 都要多保留几个数量级的特征，且相关性越强、差距越大。

Hessian 曲线贴着虚线走，大致只保留两百来个特征，而各类安全规则和启发式规则则高出一到三个数量级，动辄保留数千到数万个。差距在 $\rho = 0$ 时最小，随相关性增大而拉开，而这恰恰是竞争规则走向恶化的方向。筛得更紧，意味着优化器每一次内部求解面对的问题都小得多。

用途二：几乎精确的热启动

只靠更紧的筛选还不足以解释全部提速，热启动同样关键。因为 Hessian 热启动在活跃集不变时就是精确解，大多数路径步骤应当几乎立刻收敛。下图把这一效应单独拎出来：在两个真实数据集上，比较 Hessian 热启动与标准热启动（上一步的解）各自每一步所需的坐标下降遍数。

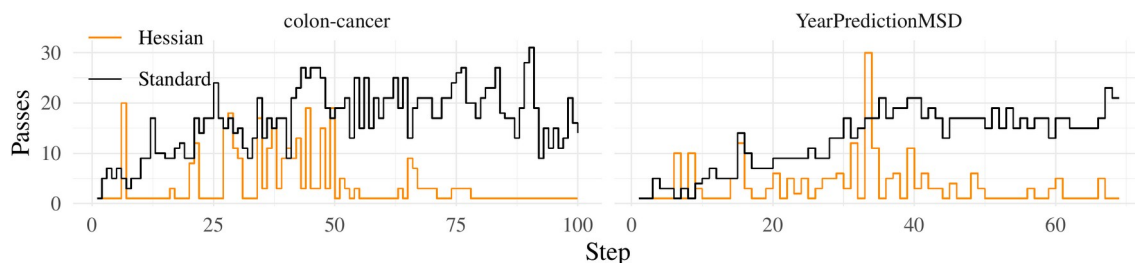


图2 colon-cancer ($n = 62$, $p = 2000$) 与 YearPredictionMSD ($n = 463715$, $p = 90$) 上每个路径步骤的坐标下降遍数：Hessian 热启动（橙）对比标准热启动（黑）。Hessian 热启动把大多数步骤压到只需一遍，因为只要活跃集不变，它就几乎是精确解。

标准热启动每一步要在 10 到 30 遍之间来回波动；Hessian 热启动则大部分步骤只用一遍，只在活跃集真正发生变化的那几步才出现尖峰。这正是该方法在 $n \gg p$ 场景下占优的机制所在：那里每一遍的成本很高，砍掉遍数的收益立竿见影。

端到端到底快多少

更紧的筛选和更好的热启动，最终能否换成对强基线的实际墙钟时间优势？下图在模拟高斯设计上，把完整路径的拟合时间与工作集策略、Celer、Blitz 相比，覆盖低维 ($n = 10000$, $p = 100$) 与高维 ($n = 400$, $p = 40000$) 两种设定、最小二乘与逻辑回归两种损失、三个相关水平。时间以每组最快方法为基准做了归一化，因此 Hessian 那根柱子固定为 1。

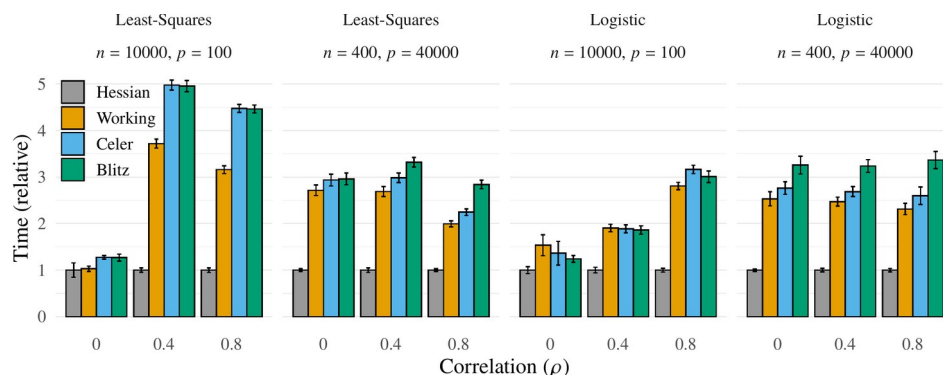


图3 模拟数据上完整路径的相对拟合时间（越低越好，Hessian 归一化为 1），跨最小二乘与逻辑回归两种损失、两种维度设定、相关水平 $\rho = 0, 0.4, 0.8$ 。Hessian 规则在每一种配置下都最快；在高相关、低维的最小二乘场景下优势最大，此时基线要慢上 3–5 $\times$ 。

Hessian 规则在每一种配置下都是最快的，而且优势最大的地方，恰恰是筛选规则历来最吃力的场景：高相关、低维的最小二乘，此时工作集策略、Celer、Blitz 都要慢上大约三到五倍。

而在相关性为零、问题本就简单时，差距收窄，这是一个诚实的信号：收益来自把困难场景处理好，而不是靠一个一刀切的常数倍加速。

真实数据上是同一个故事

这一规律延续到了十二个真实数据集上，涵盖基因表达矩阵、文本语料，以及特征多达数百万的问题。下表列出其中有代表性的一部分，报告完整路径的拟合时间（秒，越低越好）。在 ℓ_1 正则最小二乘上，Hessian 规则在全部五个数据集上都最快，且在其中除一个之外都用不到次优方法一半的时间：在 YearPredictionMSD 上是 78.8 秒对 541 秒（约 6.9 $\times$ ），在 e2006-tfidf 上是 14.3 秒对 143 秒（约 10 $\times$ ），提速接近一个数量级。

表 1 真实数据集代表性子集上完整路径的拟合时间（秒，越低越好）。Hessian 在所列的每个最小二乘集合、以及较大的逻辑回归集合上都最快，工作集策略通常是次优。在完整基准里唯一的失手是 arcene（逻辑回归），那里工作集策略略快一点。

数据集	n	p	损失	Hessian	Working	Blitz	Celer
bcTCGA	536	17,322	Least-Squares	3.00	7.67	11.7	10.6
e2006-tfidf	16,087	150,360	Least-Squares	14.3	143	277	335
YearPredictionMSD	463,715	90	Least-Squares	78.8	541	706	712
colon-cancer	62	2,000	Logistic	0.054	0.134	0.177	0.169
madelon	2,000	500	Logistic	48.2	232	240	247
rcv1	20,242	47,236	Logistic	132	266	384	378

在逻辑回归上，结论几乎一样有力，madelon（48.2 秒对 232 秒）和 rcv1（132 秒对 266 秒）都是大幅领先。十二个数据集里唯一的例外是 arcene，一个规模小、稠密的逻辑回归问题，那里工作集策略略快一线，作者选择如实报告，而不是回避这一处失手。

结论，以及它的边界

Hessian 筛选规则的吸引力在于：一个想法收获两次回报。复用二阶信息，既把筛选收紧、让求解器面对的特征大幅减少，又提供了足够准确的热启动、让许多步骤一遍即收敛。两者叠加，使它成为在本文基准上拟合 Lasso 与 ℓ_1 逻辑回归正则化路径最快的方法，而优势最大之处，正是长期以来对筛选规则最棘手的高相关场景。

边界也讲得很清楚。存储 Hessian 及其逆，比竞争方法更耗内存，当 $\min\{n, p\}$ 很大时这一开销可能变得难以承受，此时工作集策略是更稳妥的退路。而且由于这是一个为拟合整条路径而设计的序贯规则，当你只需要单个 λ 处的解时，它并非首选，那种情况下 Celer、Blitz 这类动态方法多半更优。方法本身也可以干净地推广到最小二乘和逻辑回归之外，任何可微部分二阶可微的复合目标都适用。对于在相关的高维数据上拟合整条路径、做交叉验证、再重复一遍这一最常见的工作流而言，它已经是一个足够稳妥而明确的默认选择。